

FORMATION EMBARQUÉE

TD 00 Fondamentaux

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 00 — Fondamentaux : énoncés et corrigés détaillés

Fais chaque exercice **sur papier** avant de lire le corrigé. En embarqué, ces conversions doivent devenir des réflexes.

Exercice 1 — Conversions décimal → binaire → hexa

Énoncé. Convertis 42, 100 et 200 en binaire puis en hexadécimal.

Corrigé

Méthode 1 (divisions successives par 2) : on divise par 2 en notant les restes, puis on lit les restes de bas en haut.

```
42 ÷ 2 = 21 reste 0
21 ÷ 2 = 10 reste 1
10 ÷ 2 = 5  reste 0
5 ÷ 2 = 2  reste 1
2 ÷ 2 = 1  reste 0
1 ÷ 2 = 0  reste 1    → lecture de bas en haut : 101010
```

Méthode 2 (soustraction des puissances de 2) — plus rapide de tête : $42 = 32 + 8 + 2 \rightarrow$ bits 5, 3 et 1 → **0b00101010**.

Pour l'hexa : grouper le binaire **par paquets de 4 en partant de la droite**.

Décimal	Binaire	Groupes de 4	Hexa
42	0010 1010	0010=2, 1010=A	0x2A
100	0110 0100	0110=6, 0100=4	0x64
200	1100 1000	1100=C, 1000=8	0xC8

Vérification rapide : $0x2A = 2 \times 16 + 10 = 42 \checkmark$; $0x64 = 6 \times 16 + 4 = 100 \checkmark$; $0xC8 = 12 \times 16 + 8 = 200 \checkmark$.

Exercice 2 — 0xB7 en décimal et en binaire

Énoncé. Que vaut **0xB7** en décimal ? En binaire ?

Corrigé

- Chaque chiffre hexa = 4 bits : B = **1011**, 7 = **0111** → **0b1011 0111**.
- Décimal : $B7 = 11 \times 16 + 7 = 176 + 7 = \mathbf{183}$.
- Contre-vérification par le binaire : $128 + 32 + 16 + 4 + 2 + 1 = 183 \checkmark$.

Exercice 3 — Complément à deux

Énoncé. Écris -10 en complément à deux sur 8 bits.

Corrigé

1. +10 sur 8 bits : `0000 1010`
2. Inversion de tous les bits : `1111 0101`
3. On ajoute 1 : `1111 0110`

-10 = `0b1111 0110` = 0xF6.

Vérifications : - 0xF6 lu en non signé vaut 246, et $246 = 256 - 10$ ✓ (le complément à deux, c'est « modulo 256 »). - $-10 + 10$ doit donner 0 : `1111 0110` + `0000 1010` = `1 0000 0000` → sur 8 bits, la retenue sort et il reste `0000 0000` ✓.

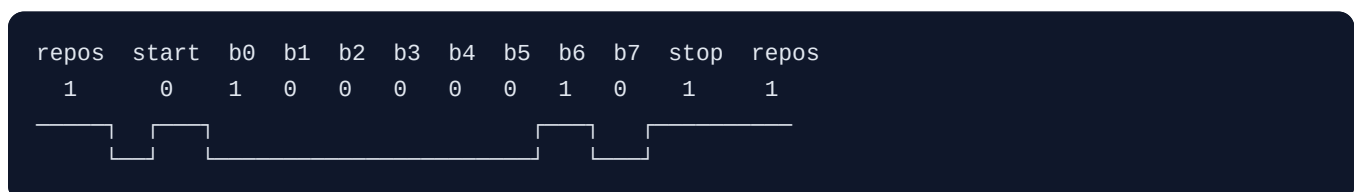
Astuce de pro : le bit de poids fort (MSB) à 1 ⇒ nombre négatif (en signé). Pour lire une valeur négative : refaire l'opération inverse (inverser + 1) et mettre un signe moins.

Exercice 4 (complément) — Trame UART : calcul de débit utile

Énoncé. Une liaison UART est configurée à 9600 bauds, 8 bits de données, sans parité, 1 bit de stop (« 8N1 »). Combien d'**octets utiles** par seconde peut-elle transporter au maximum ? Combien de temps dure la transmission du caractère `'A'` (0x41) ? Dessine la trame.

Corrigé

- Une trame 8N1 = 1 bit de start + 8 bits de données + 1 bit de stop = **10 bits transmis par octet utile.**
- 9600 bauds = 9600 bits/s → $9600 / 10 =$ **960 octets/s maximum.**
- Durée d'un bit : $1/9600 \approx$ **104,2 µs** ; durée d'une trame : $10 \times 104,2 \approx$ **1,042 ms.**
- `'A'` = 0x41 = `0b0100 0001`. L'UART envoie le **LSB en premier** :



(ligne au repos à l'état haut ; start = passage à 0 ; données LSB → MSB ; stop = retour à 1).

Ce qu'il faut retenir : à 9600 bauds, on ne transmet même pas 1 Ko/s. Pour du debug verbeux, on passe à 115200 ($\approx 11,5$ Ko/s).

Exercice 5 (complément) — Choisir le bon bus

Énoncé. Pour chaque besoin, choisis UART, I2C, SPI ou CAN, et justifie : 1. Relier 6 capteurs de température sur une carte, 2 mesures/seconde chacun. 2. Un écran TFT 320×240 à rafraîchir 20 fois par

seconde. 3. Faire dialoguer 8 calculateurs répartis dans un véhicule, milieu bruité. 4. Envoyer des messages de debug vers un PC.

Corrigé

1. **I2C** : 6 périphériques sur 2 fils seulement (chacun son adresse), et le débit requis est minuscule ($6 \times 2 \times$ quelques octets/s). SPI marcherait mais exigerait 6 lignes CS.
2. **SPI** : $320 \times 240 \text{ pixels} \times 16 \text{ bits} \times 20 \text{ Hz} \approx 24,6 \text{ Mbits/s}$ — seul SPI atteint ces débits. I2C (400 kbits/s) est $\sim 60\times$ trop lent.
3. **CAN** : conçu exactement pour ça — multi-maître, différentiel donc immunisé au bruit, arbitrage par priorité, détection d'erreurs intégrée.
4. **UART** : point à point vers le PC (via un pont USB-série), simple, universellement supporté par les terminaux série.

Exercice 6 (complément) — Lecture de registre

Énoncé. Un registre d'état 8 bits `STATUS` a cette organisation :

bit 7	bit 6	bit 5	bit 4	bits 3-2	bits 1-0
READY	ERROR	—	—	MODE (0-3)	CHANNEL (0-3)

On lit `STATUS = 0x8D`. Décode chaque champ. Puis écris (en pseudo-C) comment mettre MODE à 2 **sans toucher aux autres bits**.

Corrigé

`0x8D = 0b1000 1101`.

- READY (bit 7) = **1** → prêt.
- ERROR (bit 6) = **0** → pas d'erreur.
- MODE (bits 3-2) = **11** → **3**.
- CHANNEL (bits 1-0) = **01** → **1**.

Écriture de MODE=2 en lecture-modification-écriture :

```
uint8_t v = STATUS;
v &= ~(0x3 << 2);    // effacer les 2 bits de MODE (masque 0b0000 1100)
v |= (2 << 2);        // écrire la nouvelle valeur (0b0000 1000)
STATUS = v;
```

Point clé : ne jamais écrire directement `STATUS = 0x08` — cela écraserait CHANNEL et les autres bits. Le motif « clear puis set avec masque » est LE geste de base du bas niveau.

Auto-évaluation avant le module 01

Tu dois savoir, sans notes : convertir un octet dans les trois bases en moins d'une minute ; expliquer le complément à deux ; citer les fils et un cas d'usage de UART, I2C, SPI, CAN ; expliquer pourquoi une ISR doit être courte ; décrire les étapes de la toolchain (source → .elf → flash).